Quest 12: The Castastrophe

Lord of the Wings · covering magma with straight-line spells

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: the grid holds stacks of magma blocks; one spell clears a straight horizontal or vertical run of touching blocks (it can't jump over a gap). Find the fewest spells that clear every block.
Input: a grid of `#` (magma) and `.` (empty).
Output: the minimum total number of spells.
What changes between parts: the rule never changes, only the shapes. Part I: every stack is a rectangle. Part II: rectangles and L-shapes (any orientation and arm thickness). Part III: rectangles with a rectangular hole cut out, which can split rows and columns in two, and other stacks can sit inside a hole.

1. The story

Just before the last portal, the cursed egg takes control of Frodoff, who shoves the narrator into the Abyss of Unmarked Recursion. Waiting there is Stackrog, an old enemy who throws stacks of magma. The narrator remembers a reflection spell that bounces back one straight line of blocks at a time, and has to save energy by casting as few spells as possible while Stackrog's stacks get more and more awkward.

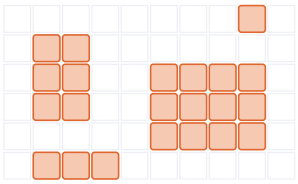
2. Understanding the question

The rules

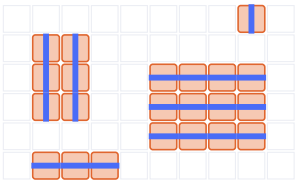
- A **stack** is a group of `#` cells joined up, down, left or right. Stacks never touch, so each can be solved on its own and the answers added up.
- A spell covers a straight run of `#` in one row or one column. It stops at the first `.`, so a row like `###..###` needs two horizontal spells.
- Every block must be covered by at least one spell. Covering a block twice is allowed, but never needed.
- A spell might as well run along the **whole** run it's in: going further never hurts. So the only choices are *which* maximal row-runs and column-runs to cast.

That last point gives the key way to look at it: every block sits on exactly one maximal horizontal run and one maximal vertical run, and **at least one of those two must be cast**.

The Part I example



the notes



7 spells (blue lines)

Extra examples of my own (all checked with your Part III code)

Stack	Spells	Why
`###` `###`	2	A 3 by 2 rectangle: 2 rows beat 3 columns.
`#####`	1	A straight line is one spell in its own direction.

`###`
`#.#`
`###`
	4	A ring: top row, bottom row, then the two middle blocks need one spell each. Columns also cost 4, because the hole splits the middle column into two runs.

`####`
`####`
`##..`
`##..`
	4	A thick L: rows cost 4, columns cost 4, and 2 columns for the left arm + 2 rows for the top arm is also 4.

Shared code: reading the grid

All three parts start the same way (only the file names differ):

```
1  import { readFileSync } from "node:fs";
2
3  const isSample = process.argv.includes("--sample");
4  const file = isSample ? "sample1.txt" : "input1.txt";
5  const INPUTS: string[][] = readFileSync(file, "utf8")
6    .trim()
7    .split("\n")
8    .map((i) => i.split(""));
9
10 const TOTAL_ROW = INPUTS.length;
11 const TOTAL_COL = INPUTS[0]?.length || 0;
```

q12/part1.ts, lines 1–11

Lines	What happens
3–4	`--sample` picks the sample file.
5–8	Trim the file, split into lines, and split each line into single characters: `INPUTS[row][col]` is `'#'` or `'.'`.
10–11	The grid size. `?.` and `|| 0` protect against an empty file.

3. Part I: rectangles only

The question

Every stack is a solid rectangle. A rectangle of height *h* and width *w* needs **min(h, w)** spells: one per row or one per column, whichever is fewer. Mixing directions never helps, because any column you don't cast leaves blocks in every row that still need a row spell.

Worked examples

Stack (from the example)	Height × width	Spells
single block at the top	1 × 1	1
left stack	3 × 2	2
right stack	3 × 4	3
bottom line	1 × 3	1
total		**7**

Extra cases: a 5 × 5 square needs 5 either way; a single 1 × 100 line needs just 1.

Step-by-step solution

1. Scan the grid for blocks whose upper and left neighbours are empty: these are the top-left corners of the rectangles.
2. From each corner, count blocks going down (height) and going right (width).
3. Add min(height, width) for each rectangle and print the total.

My solution

```ts
const stacks: string[] = [];

const isTopLeftCorner = (row: number, col: number): boolean => {
  const isAboveClear = row === 0 || INPUTS[row - 1]?.[col] === ".";
  const isLeftClear = col === 0 || INPUTS[row]?.[col - 1] === ".";

  return isAboveClear && isLeftClear;
};

const getStacks = (row: number, col: number): string => {
  let currentRow = row;
  let currentCol = col;

  let rows = 0;
  let cols = 0;

  while (currentRow < TOTAL_ROW && INPUTS[currentRow]![col] === "#") {
    currentRow++;
    rows++;
  }

  while (currentCol < TOTAL_COL && INPUTS[row]![currentCol] === "#") {
    currentCol++;
    cols++;
  }

  return `${rows}x${cols}`;
};

for (let row = 0; row < TOTAL_ROW; row++) {
  for (let col = 0; col < TOTAL_COL; col++) {
    if (INPUTS[row]![col] === "#" && isTopLeftCorner(row, col))
      stacks.push(getStacks(row, col));
  }
}

let minSpells = 0;

stacks.forEach((stack) => {
  const [row = 0, col = 0] = stack.split("x").map((s) => Number(s));

  if (row < col) minSpells += row;
  else minSpells += col;
});

console.log(minSpells);
```

q12/part1.ts, lines 13–58

Lines	What happens
15–20	`isTopLeftCorner`: a block is a rectangle's top-left corner if the cell above and the cell to the left are empty (or off the grid).
22–40	`getStacks`: from the corner, count `#` going down the column (height) and going right along the row (width). Returns them as a string like `"3x2"`.
42–47	Scan every cell; each top-left corner adds one `"HxW"` entry.
51–56	Split each string back into two numbers and add the smaller one.

Trace (sample)

Corner (row, col)	Stack string	Added
(0, 8)	`1x1`	1
(1, 1)	`3x2`	2
(2, 5)	`3x4`	3
(5, 1)	`1x3`	1
	total	**7**

> **Cost:** one pass over the grid, plus a walk down and right from each corner: about O(rows × cols). The `"3x2"` string is only used to be split straight back into numbers; storing `[3, 2]` would skip that round trip, but it works fine.

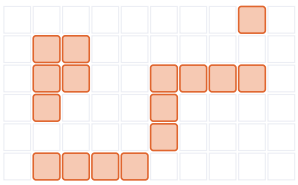
> **Only correct for rectangles (which is all Part I promises).** The code looks at just the top row and left column of each stack. On Part II's first example, a copy of this code finds the right-hand L as a `3x4` "rectangle" and prints 7 instead of 6; on `sample2.txt` it prints 21 instead of 23. That's why Part II needs a different way of finding stacks.

4. Part II: L-shapes

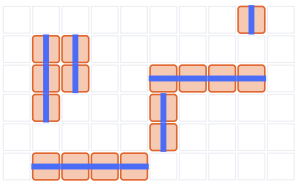
The question

Now some stacks are L-shapes: a rectangle with a rectangular piece missing from one corner. The arms can have any length and thickness, and the missing corner can be any of the four.

Worked examples

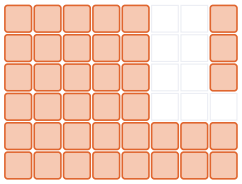


first example

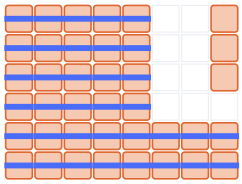


6 spells

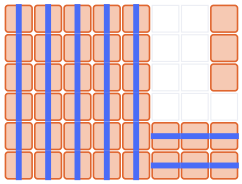
The thick L from `sample2.txt` (bounding box 8 wide × 6 tall) shows why the direction matters:



the stack



row by row: 6



5 columns + 2 rows: 7

sample2.txt stack	Spells
4 × 3 rectangle	3
thick L (8 × 6 box)	6
upper-right L	5
horizontal line	1
vertical line	1
single block	1
large lower-right L	6
total	**23**

The idea: an L has only a few sensible covers

Call the bounding box H tall and W wide, and the missing corner *missingRows* × *missingCols*. That leaves a full-height arm *verticalArm* = W − missingCols columns thick and a full-width arm *horizontalArm* = H − missingRows rows thick. The candidate covers are:

- every row: **H** spells (each row of an L is one unbroken run);
- every column: **W** spells;
- columns through the full-height arm plus rows through the full-width arm: **verticalArm + horizontalArm**.

Step-by-step solution

1. Scan the grid; at each block not seen yet, flood-fill (BFS) to collect its whole stack.
2. Work out the stack's bounding box. If the stack fills it, it's a rectangle: min(height, width).
3. Otherwise find the empty corner and measure the missing rectangle, then the thickness of the two arms.
4. Try splitting off the full-height arm first or the full-width arm first, covering each piece the cheaper way, and keep the cheaper total.
5. Add up all stacks and print.

My solution

Everything after the parsing is new.

```
const visited = new Set<string>();
const key = (row: number, col: number) => `${row},${col}`;
const isBlock = (row: number, col: number): boolean =>
  INPUTS[row]?.[col] === "#";

const getStack = (row: number, col: number): Set<string> => {
  const cells = new Set<string>();
  const queue: [number, number][] = [[row, col]];
  visited.add(key(row, col));

  while (queue.length > 0) {
    const [r, c] = queue.shift()!;
    cells.add(key(r, c));

    for (const [dr, dc] of [
      [1, 0],
      [-1, 0],
      [0, 1],
      [0, -1],
    ] as const) {
      const nr = r + dr;
      const nc = c + dc;
      if (isBlock(nr, nc) && !visited.has(key(nr, nc))) {
        visited.add(key(nr, nc));
        queue.push([nr, nc]);
      }
    }
  }

  return cells;
};

const getSpells = (cells: Set<string>): number => {
  let top = Infinity;
  let bottom = -Infinity;
  let left = Infinity;
  let right = -Infinity;

  cells.forEach((cell) => {
    const [r = 0, c = 0] = cell.split(",").map(Number);
    top = Math.min(top, r);
    bottom = Math.max(bottom, r);
    left = Math.min(left, c);
    right = Math.max(right, c);
  });

  const height = bottom - top + 1;
  const width = right - left + 1;

  if (cells.size === height * width) return Math.min(height, width);

  const corners: [number, number, number, number][] = [
    [top, left, 1, 1],
    [top, right, 1, -1],
    [bottom, left, -1, 1],
    [bottom, right, -1, -1],
  ];

```

```
71    const [cr, cc, dr, dc] = corners.find(([r, c]) => !cells.has(key(r, c)))!;
72
73    let missingRows = 0;
74    while (!cells.has(key(cr + dr * missingRows, cc))) missingRows++;
75
76    let missingCols = 0;
77    while (!cells.has(key(cr, cc + dc * missingCols))) missingCols++;
78
79    const verticalArm = width - missingCols;
80    const horizontalArm = height - missingRows;
81
82    const cutVertical =
83      Math.min(height, verticalArm) + Math.min(horizontalArm, missingCols);
84
85    const cutHorizontal =
86      Math.min(horizontalArm, width) + Math.min(missingRows, verticalArm);
87
88    return Math.min(cutVertical, cutHorizontal);
89  };
90
91  let minSpells = 0;
92
93  for (let row = 0; row < TOTAL_ROW; row++) {
94    for (let col = 0; col < TOTAL_COL; col++) {
95      if (isBlock(row, col) && !visited.has(key(row, col)))
96        minSpells += getSpells(getStack(row, col));
97    }
98  }
99
100 console.log(minSpells);
```

q12/part2.ts, lines 13–100 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
13–16	A global `visited` set of `"r,c"` keys, and `isBlock`, which is safely `false` off the grid thanks to `?.`.
18–43	`getStack`: breadth-first flood fill from one block, collecting every block of its stack.
46–60	Find the stack's bounding box: top, bottom, left, right, then its height and width.
62	If the stack fills its whole box it's a rectangle: `min(height, width)`, as in Part I.
64–71	Otherwise it's an L, so exactly one corner of the box is empty. Find it, with the direction (`dr`, `dc`) that points from that corner into the box.
73–77	Walk from the empty corner down/up its column until a block is hit: that's `missingRows`. Same along its row for `missingCols`.
79–80	Arm thicknesses: the full-height arm is `width - missingCols` columns wide; the full-width arm is `height - missingRows` rows tall.
82–83	`cutVertical`: cover the full-height arm (H × verticalArm) the cheaper way, then the rest of the other arm (horizontalArm × missingCols) the cheaper way.
85–86	`cutHorizontal`: the same, but splitting off the full-width arm first.
88	Return the cheaper of the two.
93–98	Every unvisited block starts a new stack; add its spells.

Trace (sample2.txt, real values from an instrumented copy)

Stack (top, left)	H × W	Blocks	Empty corner	missing R × C	vArm	hArm	cutV	cutH	Spells
(1, 1)	3 × 4	12	none: rectangle						3
(1, 9)	6 × 8	36	top-right	4 × 3	5	2	7	6	6
(1, 16)	6 × 7	27	bottom-left	3 × 5	2	3	5	5	5
(8, 1)	1 × 6	6	none						1
(8, 11)	5 × 1	5	none						1
(8, 17)	1 × 1	1	none						1
(8, 18)	6 × 8	38	top-left	2 × 5	3	4	7	6	6
								total	**23**

Note the stacks at (1, 9) and (1, 16) have overlapping bounding boxes (both use column 16) but never touch, so the flood fill keeps them apart, something Part I's corner trick couldn't do.

> **Is the formula always optimal for an L?** Expanding the two `min`s gives covers costing H, W, verticalArm + horizontalArm, and a few that are never cheaper than those, so it returns min(H, W, vArm + hArm). To check, I generated 2,000 random L-shapes (all four orientations, arms of any thickness) and compared your Part II code with your Part III code, which computes the true minimum: the totals matched every time.
> **Cost:** `queue.shift()` is O(length of queue), so a huge stack would make the flood fill slower than it needs to be; for these grids (about 0.06 s on the real input) it doesn't matter.

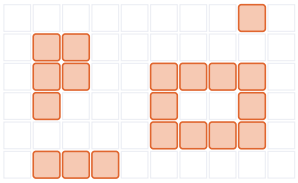
> **Fragile outside Part II's promise.** The code assumes a non-rectangle is an L.
>
> - If no corner of the box is empty (a ring, or a rectangle with a notch in one side), `corners.find(...)!` on line 71 is `undefined` and line 71 crashes: a copy run on Part III's first example stopped with *TypeError: undefined is not iterable*.
> - If more than one corner is empty (a T-shape), it silently uses the first one: for `#####` with a 2-block stem under the middle, it returns 3, but 2 spells (one row, one column) are enough.
>
> Neither happens in Part II, so your answer is fine; it's just why Part III needs a general method.

5. Part III: rectangles with holes

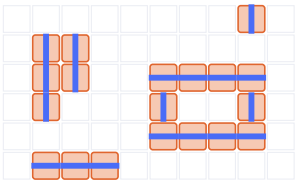
The question

Some stacks are now rectangles with a rectangular hole. A hole can break a row (or column) into two separate runs, each needing its own spell, and another stack can sit inside the hole. Shape-by-shape formulas get messy, so a general method helps.

Worked examples



first example



8 spells: the ring needs 4

Example	Stacks	Horizontal runs	Vertical runs	Answer
first example (above)	4	9	12	8
second example (7 stacks)	7	36	70	29
sample3.txt (nested)		29	93	16

All three answers were produced by your Part III code; the run counts come from an instrumented copy. Your Part III code also gives 7 on Part I's example and 6 and 23 on Part II's, so it solves all three parts.

The idea: a bipartite graph and König's theorem

Make one node for every maximal **horizontal run** and one for every maximal **vertical run**. Each block lies on exactly one of each, so draw an edge between them for each block. A set of spells covers every block exactly when it touches every edge: that's a **minimum vertex cover**. The graph is **bipartite** (edges only go horizontal ↔ vertical), and **König's theorem** says that in a bipartite graph the minimum vertex cover has the same size as the **maximum matching** (the most edges you can pick with no two sharing a node). A maximum matching can be found with augmenting paths (**Kuhn's algorithm**).

Intuition: matched edges are blocks that share no row run and no column run, so each needs its own spell. That gives a lower bound, and König says it's always reachable. Stacks inside holes need no special care: they're just more nodes and edges.

Step-by-step solution

1. Give every maximal horizontal run of blocks an id, and every maximal vertical run an id.
2. For each block, connect its horizontal run to its vertical run (a bipartite graph).
3. Find a maximum matching: for each horizontal run, look for an augmenting path (take a free vertical run, or move the current owner of one to another).
4. Print the number of matched pairs: by König's theorem it equals the fewest spells.

My solution

Everything after the parsing is new again.

```ts
const isBlock = (row: number, col: number): boolean =>
  INPUTS[row]?.[col] === "#";

const horizontalId: number[][] = INPUTS.map((line) => line.map(() => -1));
const verticalId: number[][] = INPUTS.map((line) => line.map(() => -1));

let horizontalCount = 0;
let verticalCount = 0;

for (let row = 0; row < TOTAL_ROW; row++) {
  for (let col = 0; col < TOTAL_COL; col++) {
    if (!isBlock(row, col)) continue;

    horizontalId[row]![col] = isBlock(row, col - 1)
      ? horizontalId[row]![col - 1]!
      : horizontalCount++;

    verticalId[row]![col] = isBlock(row - 1, col)
      ? verticalId[row - 1]![col]!
      : verticalCount++;
  }
}

const edges: Set<number>[] = Array.from(
  { length: horizontalCount },
  () => new Set<number>(),
);

for (let row = 0; row < TOTAL_ROW; row++) {
  for (let col = 0; col < TOTAL_COL; col++) {
    if (isBlock(row, col))
      edges[horizontalId[row]![col]!]!.add(verticalId[row]![col]!);
  }
}

const matchedTo: number[] = new Array(verticalCount).fill(-1);

const tryMatch = (horizontal: number, seen: boolean[]): boolean => {
  for (const vertical of edges[horizontal]!) {
    if (seen[vertical]) continue;
    seen[vertical] = true;

    if (matchedTo[vertical] === -1 || tryMatch(matchedTo[vertical]!, seen)) {
      matchedTo[vertical] = horizontal;
      return true;
    }
  }

  return false;
};

let minSpells = 0;

for (let horizontal = 0; horizontal < horizontalCount; horizontal++) {
  if (tryMatch(horizontal, new Array(verticalCount).fill(false))) minSpells++;
}

console.log(minSpells);
```

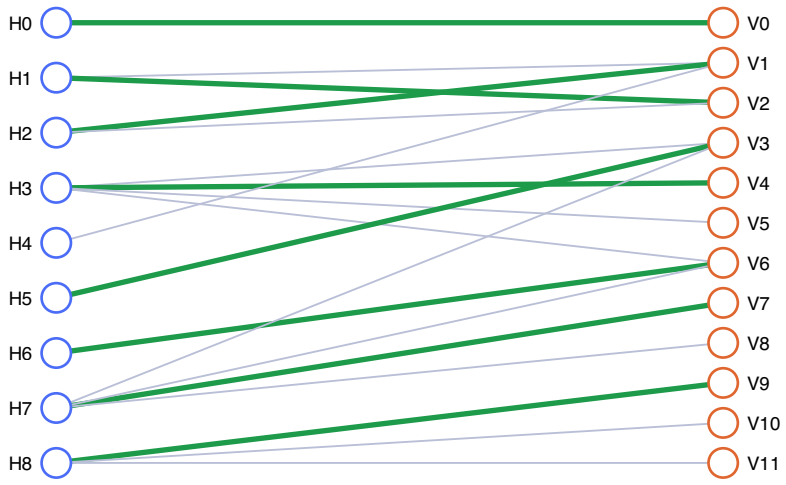
q12/part3.ts, lines 13–70 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
16–20	Two grids of run ids (–1 = empty) and two counters.
22–34	One scan, top-left to bottom-right. A block continues the horizontal run of its left neighbour if that's a block, else it starts a new one. Likewise vertically, using the block above. The left and upper cells were already visited, so their ids are ready.
36–46	`edges[h]` = the set of vertical runs that cross horizontal run `h`: one entry per block.
48	`matchedTo[v]` = which horizontal run vertical run `v` is matched to (–1 = free).
50–62	`tryMatch(h)`: try each vertical run `v` that `h` crosses. Skip ones already looked at in this search. If `v` is free, or its current partner can be moved to another vertical run (the recursive call), give `v` to `h`. This is one augmenting-path search.
66–68	Run it once for every horizontal run, with a fresh `seen` array. Each success grows the matching by one.
70	The matching size is the answer, by König's theorem.

Trace: the first example

Run ids from an instrumented copy (H = horizontal, V = vertical):

```
horizontalId                    verticalId
.  .  .  .  .  .  .  .  0  .    .  .  .  .  .  .  .  .  0  .
.  1  1  .  .  .  .  .  .  .    .  1  2  .  .  .  .  .  .  .
.  2  2  .  .  3  3  3  3  .    .  1  2  .  .  3  4  5  6  .
.  4  .  .  .  5  .  .  6  .    .  1  .  .  .  3  .  .  6  .
.  .  .  .  .  7  7  7  7  .    .  .  .  .  .  3  7  8  6  .
.  8  8  8  .  .  .  .  .  .    .  9 10 11  .  .  .  .  .  .
```



edges = blocks; green = the final matching (8 edges)

h	Crosses V…	What `tryMatch` does	Matched?
H0	0	V0 is free → take it.	yes
H1	1, 2	V1 is free → take it.	yes
H2	1, 2	V1 belongs to H1; H1 can move to V2, so H2 takes V1.	yes
H3	3, 4, 5, 6	V3 is free → take it.	yes
H4	1	V1 belongs to H2 → H2 tries V2, held by H1 → H1 has nothing left. Fails.	no
H5	3	V3 belongs to H3; H3 moves to V4, so H5 takes V3.	yes
H6	6	V6 is free → take it.	yes
H7	3, 7, 8, 6	V3: H5 has nowhere else. V7 is free → take it.	yes
H8	9, 10, 11	V9 is free → take it.	yes

Final `matchedTo = [0, 2, 1, 5, 3, -1, 6, 7, -1, 8, -1, -1]`; 8 successes, so the answer is **8**. H4 failing is the left L at work: H1, H2 and H4 (its three rows) only cross V1 and V2 (its two columns), so two vertical spells beat three horizontal ones.

> **Cost:** each `tryMatch` call can visit every edge, so O(H-runs × blocks) at worst; in practice far less. Your real input runs in about 0.07 s.
> **Harmless details:** `edges` uses a `Set`, though a horizontal and a vertical run can cross in at most one block, so there are never duplicates. The code finds the *size* of the best cover, not which spells to cast; that's all the question asks.

> **Recursion depth.** `tryMatch` calls itself once per step along an augmenting path, so on a very large grid with very long paths it could hit JavaScript's call-stack limit. Nowhere near that here.

6. Summary

Part	Shapes	How a stack is found	Spells per stack	Sample answer
I	rectangles	top-left corner, walk right and down	min(H, W)	7
II	+ L-shapes	BFS flood fill + bounding box	min of two arm splits	23
III	+ rectangles with holes, nested	not needed: one global graph	maximum bipartite matching	16

Key ideas: each block forces "its row run or its column run", which makes this a **minimum vertex cover** on a **bipartite graph**. **König's theorem** turns that into a **maximum matching**, found with **augmenting paths** (Kuhn's algorithm). For simple shapes a direct formula works, but it breaks as soon as the shapes change.